

Documento Técnico — Endpoint Investigator

Problema

O trabalho pede uma ferramenta que **colete, relacione e interprete** o estado de um endpoint GNU/Linux cobrindo **processos, permissões e serviços**, com pelo menos duas correlações, e que separe **evidência, interpretação, hipótese e evidência ausente** — dizendo com clareza quando a evidência é insuficiente.

O risco pedagógico central é o raciocínio simplista do tipo "serviço root = vulnerabilidade" ou "conexão externa = C2". A ferramenta precisa, portanto, tratar cada indicador como *contexto*, e só concluir risco quando várias fontes independentes se sustentam mutuamente. A coleta (esta metade) existe para dar à análise uma base de fatos estruturada, rastreável e honesta sobre suas lacunas.

Arquitetura

O pipeline é linear e tem **um módulo por etapa** (ver o diagrama no README). Duas entradas distintas — um dataset no formato do professor e a máquina ao vivo — convergem para **um único Snapshot normalizado**. A análise (correlação, evidências, relatório) lê sempre esse mesmo contrato, sem saber de onde os dados vieram.

O Snapshot reúne:

- `processes{pid→Process}` com identidade (UID real/efetivo no live), estado, linha de comando, executável, script (quando o executável é interpretador), **cadeia de ancestralidade** (`ancestry`) e a unidade `systemd` mapeada;
- `services{unit→Service}` com usuário, `ExecStart`, `MainPID`, arquivo de unidade e os caminhos referenciados;
- `files{path→FileInfo}` com dono/grupo/modo, flags derivadas (`world/group-writable`, `SUID`, `SGID`) e a conclusão `nonroot_writable` com justificativa;
- `logs[]` estruturados e uma **linha do tempo** tipada e ordenada (`timeline`);
- `sockets[]` e `cron[]` (extras do modo live);
- `gaps[]` — tudo o que **não** pôde ser coletado.

Cada fato carrega `src` (proveniência: `processes.csv:5, /proc/2417/status, ...`), o que permite que toda conclusão da análise aponte para a observação que a gerou.

Decisões técnicas

- **Python 3.10+, apenas biblioteca padrão.** O Kali já traz Python; não há o que instalar, e o gerador do professor também é Python.
- **Um snapshot sob demanda, sem agente.** O enunciado permite; simplifica.
- **Sem LLM dentro da ferramenta.** Toda a análise é baseada em regras e rastreável, evitando o anti-padrão "DADOS → LLM". O uso de IA no desenvolvimento é documentado à parte.
- **Comparação temporal em wall-clock *naïve*.** Os CSVs anexam `-03:00` a horários UTC; o journal traz o mesmo wall-clock sem offset. Normalizamos tudo para wall-clock sem fuso, de modo que `mtime`, logins e reinícios sejam diretamente comparáveis (essencial para a reconstrução temporal C4).
- **`nonroot_writable` como conceito central de permissão.** Um arquivo conta como gravável por não-root quando: `o+w`; ou o grupo tem escrita e não é root (no live checa-se a participação via `grp/pwd`; no dataset assume-se que o grupo pode conter não-root, e isso é dito na justificativa); ou o dono não é root; ou **qualquer diretório-pai** satisfaz uma dessas condições (o arquivo pode ser substituído). Usa-se `None` quando não há dados de permissão.
- **Mapeamento processo → serviço por métodos em ordem**, registrando qual casou: `cgroup`, `MainPID` (live), `cmd == ExecStart` com `ppid 1`, mesmo script, herança de ancestral mapeado e, por fim, nome-base da unidade == `basename` do executável (fraco). Registrar o método torna a força da associação auditável.
- **Coleta de arquivos por contexto, não por varredura.** No live só são inspecionados os caminhos referenciados por serviços, processos e cron, cada um com todos os diretórios-pai até `/`. Isso mantém a coleta barata e relevante.

- **Correção no gerador do professor.** `random_noise()` reaproveitava o dict já construído por `scenario_normal()` e o reenviava a `build()`, quebrando com `ValueError: too many values to unpack` em `--level intermediate/challenge`. Passou a partir dos dados crus (tuplas). Detalhes em `report_galazzi.md`.

Limitações da coleta

A ferramenta **declara** estas limitações em vez de escondê-las (elas aparecem em gaps e no relatório):

- **Snapshot único.** Processos efêmeros não são vistos; logs podem ter sido rotacionados ou limpos; `mtime` pode ser forjado (`touch`).
- **Modo dataset.**
 - Sem UID/EUID nem participação em grupos.
 - Sem crontabs/timers.
 - Horários sem ano e fuso ambíguo (tratados via wall-clock naive).
 - O mapeamento processo ↔ serviço depende de casamento de comando.
- **Modo live.** Requer root para visão completa; cada permissão ou log que não pôde ser lido vira uma lacuna explícita. Sem root, a visibilidade é parcial e isso é declarado.

Estratégia de investigação

A análise parte de uma pergunta por achado: *que relação entre fontes diferentes torna este fato relevante?* Um fato isolado — serviço root, arquivo 0777, `curl` em execução — nunca é achado. O que se investiga é a combinação **identidade privilegiada + recurso utilizado + capacidade de modificação**, exatamente o raciocínio de exemplo do enunciado.

Cada achado é registrado em quatro campos separados na própria estrutura de dados (Finding): **evidência** (fatos observados, cada um com `src`), **interpretação** (o significado técnico), **hipótese** (o que isso pode explicar) e **evidência ausente** (o que confirmaria ou rejeitaria a hipótese). A ferramenta tem quatro conclusões, e duas delas existem para *não* acusar: `INCONCLUSIVO`, quando falta evidência, e `CONTEXTO_OK`, quando a relação foi verificada e está correta. Um serviço root com script 0700 aparece no relatório como verificação sem achado — é assim que se demonstra que "root" não é o gatilho.

A confiança é o número de tipos de fonte independentes que sustentam a relação; `RISCO` exige pelo menos dois. O que não pôde ser avaliado é dito: binário de sistema sem permissão coletada sai como "não avaliado", nunca como "ok".

Principais correlações

- **C1 — serviço + processo + permissão.** Para cada serviço ativo como root, localiza o processo que o executa (só mapeamento forte: `cgroup`, `MainPID`, `ExecStart`, `script`) e avalia cada caminho referenciado. `nonroot_writable` verdadeiro → `RISCO`, com a cadeia serviço → root → PID → arquivo → modo → quem pode gravar; o diretório-pai gravável conta, e é ele que vira evidência. Permissão ausente em caminho fora do padrão → `INCONCLUSIVO`.
- **C2 — processo + PPID + usuário.** Reconstrói a cadeia de ancestralidade com a identidade de cada elo. A direção importa: root → usuário é queda de privilégio (normal); usuário → root sem `sudo/su/pkexec` na cadeia é `RISCO`. Filho de serviço root com destino externo em `argv` é `INCONCLUSIVO`, citando o log do próprio serviço quando ele descreve a atividade — o nome `curl` não dispara nada.
- **C3 — arquivo + usuário + uso.** `SUID` fora dos diretórios padrão é `INCONCLUSIVO`, informando se algum processo ou serviço o usa; arquivo gravável por todos sem vínculo com execução privilegiada é `CONFIG_INADEQUADA`.
- **C4 — processo + serviço + log.** Ordena login não-root, `mtime` e início do serviço. Em `tampered_after_login` a sequência login → modificação → início entra no achado de C1; em `correlation` o `mtime` é anterior ao login, e a ferramenta diz que não há indício de modificação durante a sessão.

Dois ajustes em relação ao plano inicial: o mínimo de duas fontes vale só para RISCO (exigir duas fontes para dizer "não sei" seria contraditório), e ssh/cron nos datasets saem como "não avaliado", pois suas permissões não foram coletadas. A integração com o modo live corrigiu dois falsos positivos da coleta: link simbólico lido com modo 0777 e diretório com *sticky bit* (/tmp) tratado como substituível.

Uso de IA

IA generativa (Claude, via Claude Code) foi usada como apoio durante todo o desenvolvimento: planejamento da arquitetura, escrita e revisão de código e testes, e redação desta documentação. As regras foram validadas por testes automatizados sobre os nove cenários do gerador e por execução no modo live. **A ferramenta em si não usa modelo de linguagem**: coleta, normalização e correlação são regras determinísticas, e toda conclusão aponta para a evidência que a gerou. Não há etapa "DADOS → LLM".

Limitações da análise

- **Condições, não incidentes.** Nenhum achado atribui autoria ou prova exploração; coincidência temporal é descrita como coincidência.
- **Falsos positivos.** Escrita de grupo conta como gravável por não-root no dataset (não há membros de grupo); um processo root filho de não-root pode vir de mecanismo legítimo que o snapshot não mostra.
- **Falsos negativos.** Arquivo gravável por todos pode ser executado por cron, timer ou unidade inativa ausente da coleta; processos efêmeros não são vistos.
- **Heurísticas.** Destino externo é lido de argv; a ligação log ↔ atividade usa palavras-chave. mtime e logs podem ser forjados ou rotacionados.
- **Fora do escopo:** hashes e integridade de pacotes, ACLs e *capabilities*, SELinux/AppArmor, containers, rootkits de kernel (a ferramenta confia em /proc).